JAVA PROGRAMMING

UNIT 1 – INTRODUCTION TO JAVA

1. Java's Magic: The Bytecode

Bytecode is a highly optimized set of instructions designed to be executed by what is called the Java Virtual Machine (JVM), which is part of the Java Runtime Environment (JRE). In essence, the original JVM was designed as an interpreter for bytecode.

Java follows a compilation-and-execution model in which source code is translated into an intermediate form called bytecode. The Java compiler converts a .java file into a .class file containing bytecode. The Java Virtual Machine (JVM) loads and executes that bytecode.

```
Java Source Program (.java)
          ↓
      Java Compiler
        javac
          ↓
    Bytecode (.class)
          ↓
         JVM
          ↓
 Operating-system-specific execution
```

- JDK (Java Development Kit): tools required to develop Java programs; it includes the compiler and runtime-related tools.
- JRE (Java Runtime Environment): environment needed to run Java applications.
- JVM (Java Virtual Machine): Interprets and executes Java bytecode.
- Bytecode is designed to be platform independent.
- The JVM may interpret bytecode and/or use Just-In-Time (JIT) compilation to execute frequently used code efficiently.
- The source program is therefore not directly compiled into one particular processor's machine code.

Example

```
// Hello.java
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

Compile: javac Hello.java

Run: java Hello

Compilation creates Hello.class. The JVM loads the class and invokes main().

2. Java Buzzwords / Important Features

Simple: Java is easy to learn and use, especially for programmers familiar with C or C++.

Object-Oriented: Java uses objects and supports important OOP concepts like inheritance, polymorphism, and encapsulation.

Robust: Java provides strong type checking, exception handling, and automatic memory management through garbage collection.

Multithreaded: Java allows multiple tasks to run at the same time using threads.

Architecture-Neutral: Java programs are converted into bytecode that can run on any system with a JVM.

High Performance: Java uses Just-In-Time (JIT) compilation to improve program execution speed.

Distributed: Java supports network programming and technologies such as TCP/IP and RMI.

Dynamic: Java can load and link classes at runtime, making programs flexible and adaptable.

3. Object-Oriented Programming

Object-oriented programming represents a problem using objects that combine data and operations. A class defines the structure and behavior; an object is a runtime instance of the class.

Major OOP Concepts

- **Encapsulation**: binding data and methods together and restricting direct access to internal data.
- **Abstraction**: showing essential operations while hiding unnecessary implementation details.
- **Inheritance**: deriving a new class from an existing class.
- **Polymorphism**: allowing the same operation or interface to behave differently in different situations.

Simple Encapsulation Example

```
class Student {
    private int marks;

    public void setMarks(int marks) {
        if (marks >= 0 && marks <= 100)
            this.marks = marks;
    }

    public int getMarks() {
        return marks;
    }
}
```

Here marks is private. Outside code cannot directly write student.marks. It must use the public methods. The setter also validates the value. This is a basic example of encapsulation.

4. First Simple Java Program

```
class FirstProgram {
    public static void main(String[] args) {
        System.out.println("Welcome to Java");
```

```
    }
}
```

Part	Purpose
class FirstProgram	Declares a class named FirstProgram.
Public	Makes the member accessible from outside its class.
Static	Allows main() to be invoked without creating an object.
Void	Indicates that main() returns no value.
Main	Standard entry point of a Java application.
String[] args	Receives command-line arguments.
System.out.println()	Displays text followed by a new line.

PRIMITIVE TYPES, VARIABLES, CONVERSION AND ARRAYS

5. Primitive Data Types

Type	Size	Example	Use
byte	8-bit	byte b=10;	Small integers
short	16-bit	short s=1000;	Small/medium integers
int	32-bit	int n=100;	Common integer calculations
long	64-bit	long n=100000L;	Large integer values
float	32-bit	float x=2.5f;	Single-precision decimal values
double	64-bit	double x=2.5;	Double-precision decimal values
char	16-bit	char ch='A';	Single Unicode character
boolean	true/false	boolean flag=true;	Logical conditions

6. Integer Types

byte, short, int and long represent signed integer values. Among them, int is the normal choice for many integer calculations.

```
class IntegerDemo {
    public static void main(String[] args) {
        byte b = 10;
        short s = 2000;
        int i = 50000;
        long l = 9000000000L;

        System.out.println(b);
        System.out.println(s);
        System.out.println(i);
        System.out.println(l);
    }
}
```

7. Floating-Point Types

float and double represent floating-point values. A decimal literal is normally treated as double; the f suffix is used for float.

```
class FloatDemo {
    public static void main(String[] args) {
        float f = 12.5f;
        double d = 12345.6789;

        System.out.println("Float = " + f);
        System.out.println("Double = " + d);
    }
}
```

8. Character Type

char represents one 16-bit Unicode character and uses single quotation marks.

```
class CharDemo {
    public static void main(String[] args) {
        char ch = 'A';
        char unicode = '\u0041';

        System.out.println(ch);
        System.out.println(unicode);
    }
}
```

9. Boolean Type

```
class BooleanDemo {
    public static void main(String[] args) {
        int marks = 75;
        boolean passed = marks >= 40;

        System.out.println("Passed = " + passed);
    }
}
```

10. Variables

A variable is a named storage location associated with a data type. Java variables may be local variables, instance variables or static/class variables.

Variable	Declared where	Association
Local	Inside a method/block	Particular method invocation
Instance	Inside class but outside methods	Each object has its own copy
Static	With static keyword inside class	One class-level copy, associated with the class

```
class VariableDemo {
    static int collegeCode = 101;  // static variable
    int rollNo = 10;               // instance variable

    void show() {
        int marks = 90;            // local variable
        System.out.println(collegeCode);
        System.out.println(rollNo);
        System.out.println(marks);
    }
}
```

11. Type Conversion and Casting

Java's Automatic Conversions When one type of data is assigned to another type of variable, an automatic type conversion will take place if the following two conditions are met:

- The two types are compatible.
- The destination type is larger than the source type.

When these two conditions are met, a widening conversion takes place. For example, the int type is always large enough to hold all valid byte values, so no explicit cast statement is required.

Widening Conversion

A value can be converted automatically to a compatible type capable of representing a wider range of values.

```
class WideningDemo {
    public static void main(String[] args) {
        int n = 25;
        double d = n;

        System.out.println("int = " + n);
        System.out.println("double = " + d);
    }
}
```

Narrowing Conversion / Casting

Converting a larger numeric type to a smaller type normally requires an explicit cast. Information can be lost.

```
class CastingDemo {
    public static void main(String[] args) {
        double x = 25.75;
        int y = (int) x;

        System.out.println("Original = " + x);
        System.out.println("After cast = " + y);
    }
}
```

Output: 25.75 and 25. The fractional part is discarded.

12. One-Dimensional Array

An array stores a fixed number of elements of the same type. Array indexing starts from 0.

A one-dimensional array is, essentially, a list of like-typed variables. To create an array, you first must create an array variable of the desired type. The general form of a one-dimensional array declaration is

```
type var-name[ ];
```

```
class ArrayDemo {
    public static void main(String[] args) {
        int[] marks = {80, 75, 90, 85, 70};

        int sum = 0;

        for (int i = 0; i < marks.length; i++) {
            System.out.println("marks[" + i + "] = " + marks[i]);
            sum += marks[i];
        }

        double average = (double) sum / marks.length;
        System.out.println("Sum = " + sum);
        System.out.println("Average = " + average);
    }
}
```

Important Array Points

- Declaration: int[] a;
- Creation: a = new int[5];
- Initialization: int[] a = {10,20,30,40};
- Index range for an array of size n is 0 to n-1.
- length gives the number of elements.
- Accessing an invalid index causes ArrayIndexOutOfBoundsException.

CLASSES, OBJECTS AND METHODS

13. Fundamentals of a Class

A class is a blueprint for objects. It can contain fields, methods, constructors, static members and other declarations. A class is declared by use of the class keyword. A simplified general form of a class definition is shown here:

```
class classname {

 type instance-variable1;

type instance-variable2;

 // ...

 type instance-variableN;

 type methodname1(parameter-list) {

 // body of method

 }

 type methodname2(parameter-list) {

 // body of method

 }

 // ...
```

```
 type methodnameN(parameter-list) {

 // body of method

 }

}
```

The data, or variables, defined within a class are called instance variables. The code is contained within methods. Collectively, the methods and variables defined within a class are called members of the class.

Example 1:

```
class Student {
    int usn;
    String name;

    void display() {
        System.out.println("USN  : " + usn);
        System.out.println("Name : " + name);
    }
}
```

Example 2:

```
class Box {
 double width;
 double height;
 double depth;
}

// This class declares an object of type Box.

class BoxDemo {
  public static void main(String args[]) {
  Box mybox = new Box();
  double vol;
  // assign values to mybox's instance variables
  mybox.width = 10;
  mybox.height = 20;
  mybox.depth = 15;
 // compute volume of box
 vol = mybox.width * mybox.height * mybox.depth;
 System.out.println("Volume is " + vol);
 }

}
```

14. Declaring and Creating Objects

```
class StudentDemo {
    public static void main(String[] args) {
        Student s1 = new Student();

        s1.usn = 101;
        s1.name = "Anu";

        s1.display();
```

```
    }
}
```

Student s1 declares a reference variable. new Student() creates an object. s1 stores a reference to that object.

15. Assigning Object Reference Variables

Consider the below example-

```
Box b1 = new Box();

Box b2 = b1;
```

b1 and b2 will both refer to the same object. The assignment of b1 to b2 did not allocate any memory or copy any part of the original object. It simply makes b2 refer to the same object as does b1. Thus, any changes made to the object through b2 will affect the object to which b1 is referring, since they are the same object. This situation is depicted here: Although b1 and b2 both refer to the same object, they are not linked in any other way. For example, a subsequent assignment to b1 will simply unhook b1 from the original object without affecting the object or affecting b2.

For example:

```
 Box b1 = new Box();

Box b2 = b1; // ... b1 = null;
```

Here, b1 has been set to null, but b2 still points to the original object.

```
class ReferenceDemo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Anu";

        Student s2 = s1;
        s2.name = "Ravi";

        System.out.println(s1.name);
        System.out.println(s2.name);
    }
}
```

Both references refer to the same object. The output is Ravi for both references. Assigning an object reference does not create a second independent object.

16. Methods

A method defines an operation performed by a class or object. A method must be declared within a class. It is defined with the name of the method, followed by parentheses **()**. When a method specifies a return type other than void, it must use the return keyword to pass data back.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
```

```
    void display() {
        System.out.println("Calculator object");
    }
}

class MethodDemo {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        int result = c.add(10, 20);
        System.out.println("Sum = " + result);

        c.display();
    }
}
```

17. Constructors

A constructor is a special member used to initialize a newly created object. It has the same name as the class and no return type. A constructor initializes an object immediately upon creation. It has the same name as the class in which it resides and is syntactically similar to a method. Once defined, the constructor is automatically called when the object is created, before the new operator completes. Constructors look a little strange because they have no return type, not even void. This is because the implicit return type of a class' constructor is the class type itself.

Example 1:

```
class Student {
    int usn;
    String name;

    Student(int usn, String name) {
        this.usn = usn;
        this.name = name;
    }

    void display() {
        System.out.println(usn + " " + name);
    }
}

class ConstructorDemo {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Anu");
        Student s2 = new Student(102, "Ravi");

        s1.display();
        s2.display();
    }
}
```

Example 2:

```
class Box {
 double width;
double height;
```

```
double depth;
 // This is the constructor for Box.
 Box() {
System.out.println("Constructing Box");
width = 10;
height = 10;
depth = 10;
 }
// compute and return volume
 double volume() {
       return width * height * depth;
}
}
class BoxDemo {
       public static void main(String args[])
{
                                    // declare, allocate, and initialize Box objects
 Box mybox1 = new Box();
 Box mybox2 = new Box();
 double vol;                        // get volume of first box
 vol = mybox1.volume();
 System.out.println("Volume is " + vol);
 // get volume of second box
 vol = mybox2.volume();
 System.out.println("Volume is " + vol);
 }
}
```

O/P:

```
 Constructing Box
 Constructing Box
 Volume is 1000.0
 Volume is 1000.0
```

Constructor Overloading

constructor overloading is a technique that allows a class to have more than one constructor, provided each constructor has a different parameter list.

Just like method overloading, Java distinguishes between different constructors based on the number, data types, or order of the arguments passed when you create a new object. This allows you to initialize objects in different ways depending on what data you have available.

```
class Box {
    int length, breadth;

    Box() {
        length = breadth = 0;
    }

    Box(int side) {
        length = breadth = side;
```

```
    }

    Box(int length, int breadth) {
        this.length = length;
        this.breadth = breadth;
    }

    void area() {
        System.out.println("Area = " + length * breadth);
    }
}
```

Example 2:

```
class Box {
 double width;
 double height;
 double depth;
 // constructor used when all dimensions specified
       Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
 }
 // constructor used when no dimensions specified
       Box() {
        width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
 }
 // constructor used when cube is created
       Box(double len) {
        width = height = depth = len;
 }
 // compute and return volume
       double volume() {
       return width * height * depth;
 }
}
class OverloadCons {
 public static void main(String args[]) {
 // create boxes using the various constructors
 Box mybox1 = new Box(10, 20, 15);
 Box mybox2 = new Box();
```

```
 Box mycube = new Box(7);
 double vol;
 // get volume of first box
 vol = mybox1.volume();
 System.out.println("Volume of mybox1 is " + vol);
 // get volume of second box
 vol = mybox2.volume();
 System.out.println("Volume of mybox2 is " + vol);

// get volume of cube
 vol = mycube.volume();
 System.out.println("Volume of mycube is " + vol);
 }
}
```

Output:

```
 Volume of mybox1 is 3000.0
 Volume of mybox2 is -1.0
 Volume of mycube is 343.0
```

18. this Keyword

this is a reference to the current object. It is especially useful when an instance variable and a parameter have the same name. Sometimes a method will need to refer to the object that invoked it. To allow this, Java defines the 'this' keyword. this can be used inside any method to refer to the current object. That is, this is always a reference to the object on which the method was invoked.

```
class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println(id + " " + name);
    }
}
```

- this.id refers to the instance variable.
- id refers to the constructor parameter.
- this can also be used to invoke another instance method.
- this() invokes another constructor in the same class and must be the first statement in a constructor.

19. Garbage Collection

Java automatically manages heap memory. An object becomes eligible for garbage collection when it is no longer reachable through a live reference. Java takes a different approach; it handles deallocation for you automatically. The technique that accomplishes this is called garbage collection. It works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed. There is no explicit need to destroy objects as in C++. Garbage collection only occurs sporadically (if at all) during the execution of your program. It will not occur simply because one or more objects exist that are no longer used.

```
class GCDemo {
    public static void main(String[] args) {
        GCDemo obj = new GCDemo();

        obj = null;           // object can become eligible for GC

        System.gc();          // request to JVM; not a guarantee
        System.out.println("End of program");
    }
}
```

Garbage collection is controlled by the JVM. Calling System.gc() only requests collection; the programmer cannot force a particular object to be collected at a particular instant.

20. A Stack Class

OOP's most important benefits is the encapsulation of data and the code that manipulates that data. As you have seen, the class is the mechanism by which encapsulation is achieved in Java. By creating a class, you are creating a new data type that defines both the nature of the data being manipulated and the routines used to manipulate it. Further, the methods define a consistent and controlled interface to the class' data. Thus, you can use the class through its methods without having to worry about the details of its implementation or how the data is actually managed within the class. Stack class defines two data items and three methods. The stack of integers is held by the array stack. This array is indexed by the variable tos, which always contains the index of the top of the stack. The Stack( ) constructor initializes tos to –1, which indicates an empty stack. The method push( ) puts an item on the stack. To retrieve an item, call pop( ). Since access to the stack is through push( ) and pop( ). Peek() method returns the top most item of the stack.

```
class TestStack {

      public static void main(String args[]) {

      Stack mystack1 = new Stack();

      Stack mystack2 = new Stack();

      for(int i=0; i<10; i++) mystack1.push(i);

      for(int i=10; i<20; i++) mystack2.push(i);

            System.out.println("Stack in mystack1:");

      for(int i=0; i<10; i++)

            System.out.println(mystack1.pop());

      System.out.println("Stack in mystack2:");

      for(int i=0; i<10; i++)

      System.out.println(mystack2.pop());

    }

}
```

Output:

```
Stack in mystack1:
9
 8
 7
 6
 5
 4
 3
 2
 1
 0
 Stack in mystack2:
 19
 18
 17
 16
 15
 14
 13
 12
 11
 10
```

METHODS AND CLASSES

26. Method Overloading

Method overloading means defining multiple methods with the same name in the same class but with different parameter lists. It is compile-time polymorphism.

When an overloaded method is invoked, Java uses the type and/or number of arguments as its guide to determine which version of the overloaded method to actually call. Thus, overloaded methods must differ in the type and/or number of their parameters. While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method. When Java encounters a call to an overloaded method, it simply executes the version of the method whose parameters match the arguments used in the call.

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
        System.out.println(c.add(2.5, 3.5));
        System.out.println(c.add(1, 2, 3));
    }
}
```

Example 2:

```
class OverloadDemo {
 void test() {
 System.out.println("No parameters");
 }
 // Overload test for one integer parameter.
 void test(int a) {
 System.out.println("a: " + a);
 }
```



```
 // Overload test for two integer parameters.
 void test(int a, int b) {
       System.out.println("a and b: " + a + " " + b);
 }
```

```
 // Overload test for a double parameter
 double test(double a) {
       System.out.println("double a: " + a);
       return a*a;
 }
}
class Overload {
        public static void main(String args[]) {
       OverloadDemo ob = new OverloadDemo();
       double result;
       // call all versions of test()
       ob.test();
        ob.test(10);
       ob.test(10, 20);
       result = ob.test(123.25);
        System.out.println("Result of ob.test(123.25): " + result);
 }
}
```

output:

```
 No parameters
 a: 10
 a and b: 10 20
 double a: 123.25
 Result of ob.test(123.25): 15190.5625
```

27. Using Objects as Parameters

A method can accept an object reference as a parameter. This is useful when operations depend on another object's state.

```
class Student {
    int marks;

    Student(int marks) {
        this.marks = marks;
    }

    void compare(Student other) {
        if (this.marks > other.marks)
            System.out.println("Current student has higher marks");
        else if (this.marks < other.marks)
            System.out.println("Other student has higher marks");
        else
            System.out.println("Both have equal marks");
    }
}

class ObjectParameterDemo {
    public static void main(String[] args) {
        Student s1 = new Student(85);
        Student s2 = new Student(90);
```

```
        s1.compare(s2);
    }
}
```

28. Argument Passing in Java

Java is strictly pass-by-value. For an object parameter, the value being copied is the reference. This distinction is important.

Primitive Argument – Call by value

```
class PassPrimitive {
    static void change(int x) {
        x = 100;
    }

    public static void main(String[] args) {
        int a = 10;
        change(a);
        System.out.println(a);
    }
}
```

Output is 10 because the method receives a copy of a.

Object Reference Argument – Call by reference

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}

class PassObject {
    static void changeName(Student s) {
        s.name = "Ravi";
    }

    public static void main(String[] args) {
        Student s1 = new Student("Anu");

        changeName(s1);

        System.out.println(s1.name);
    }
}
```

Output is Ravi because the copied reference still refers to the same Student object, whose state is changed.

29. Returning Objects

```
class Box {
    int value;

    Box(int value) {
        this.value = value;
    }
```

```
    Box larger(Box b) {
        if (this.value >= b.value)
            return this;
        else
            return b;
    }
}

class ReturnObjectDemo {
    public static void main(String[] args) {
        Box b1 = new Box(50);
        Box b2 = new Box(80);

        Box result = b1.larger(b2);

        System.out.println("Larger value = " + result.value);
    }
}
```

30. Access Control

Access control in Java is managed using four access modifiers that regulate the visibility and accessibility of classes, methods, constructors, and fields. These modifiers play a central role in enforcing encapsulation.

- **Public modifier:** Accessible from anywhere in the program
- **Protected modifier:** Accessible within the same package and by subclasses
- **Private modifier:** Accessible only within the same class
- **Default modifier:** Accessible only within the same package

Modifier	Same class	Same package	Subclass in another package	Other packages
private	Yes	No	No	No
default	Yes	Yes	Package dependent	No
protected	Yes	Yes	Yes, through inheritance rules	No
public	Yes	Yes	Yes	Yes

Access control is a major part of encapsulation. A common design is to keep data private and provide carefully designed public methods.

31. static Keyword

A static member belongs to the class rather than to an individual object. Static variables are shared among objects of the class. It is possible to create a member that can be used by itself, without reference to a specific instance. To create such a member, precede its declaration with the keyword static. When a member is declared static, it can be accessed before any objects of its class are created, and without reference to any object.

```
class Student {
    private int rollNo;
```

```
    private static int count = 0;

    Student(int rollNo) {
        this.rollNo = rollNo;
        count++;
    }

    public static int getCount() {
        return count;
    }
}

class StaticDemo {
    public static void main(String[] args) {
        Student s1 = new Student(101);
        Student s2 = new Student(102);
        Student s3 = new Student(103);

        System.out.println("Objects created = " + Student.getCount());
    }
}
```

Output: Objects created = 3.

- Static variable: one shared copy per class.
- Static method: can be called using the class name.
- Static methods do not have an implicit this reference.
- A static method can directly access static members, but cannot directly access instance members.

32. final Keyword

In Java, the final keyword is used to restrict changes and make code more secure and predictable. It can be applied to variables, methods, and classes to prevent modification, overriding, or inheritance. This helps in creating constant values, stable methods, and immutable classes.

- Final variable cannot be changed once assigned
- Final method cannot be overridden
- Final class cannot be inherited

final Variable

```
class FinalVariable {
    public static void main(String[] args) {
        final int MAX_MARKS = 100;

        System.out.println(MAX_MARKS);

        // MAX_MARKS = 90;  // Compile-time error
    }
}
```

33. Command-Line Arguments

Command-line arguments in Java are values passed to a program during execution through the command prompt. These arguments are stored as String values in the main() method parameter.

- Used to provide input without hardcoding values in the program.

- Helps make programs flexible and reusable for different inputs.
- Commonly used for file names, configuration values, and user data at runtime.
- Command-line arguments are passed after the class name while running the Java program. C:\>java programName arguments

```
class CommandLineDemo {
    public static void main(String[] args) {
        System.out.println("Number of arguments = " + args.length);

        for (int i = 0; i < args.length; i++) {
            System.out.println("args[" + i + "] = " + args[i]);
        }
    }
}
```

Example command: java CommandLineDemo Java Programming

args[0] contains Java and
args[1] contains Programming.

All command-line arguments arrive as strings.

Example 2:

```
class AddNumbers {
    public static void main(String[] args) {

        if (args.length < 2) {
            System.out.println("Please provide two numbers.");
            return;
        }

        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);

        System.out.println("Sum = " + (a + b));
    }
}
```

Example: java AddNumbers 10 20 → Sum = 30.